

5.2 C Program – Linear Probing

```
#include <stdio.h>
#define SIZE 11
void linearProbing(int a[], int n)
{
    int table[SIZE];
    int i, key, index, pos;
    for (i = 0; i < SIZE; i++)
        table[i] = -1;
    for (i = 0; i < n; i++)
    {
        key = a[i];
        index = key % SIZE;
        pos = index;
        while (table[pos] != -1)
            pos = (pos + 1) % SIZE;
        table[pos] = key;
    }
    printf("\nLinear Probing:\n");
    for (i = 0; i < SIZE; i++)
        printf("Index %d : %d\n", i, table[i]);
}
int main()
{
    int a[] = {23, 43, 13, 33, 53, 63, 73};
    linearProbing(a, 7);
    return 0;
}
```

Output

```
Linear Probing:
Index 0 : 33
Index 1 : 23
Index 2 : 13
Index 3 : -1
Index 4 : -1
Index 5 : -1
Index 6 : -1
Index 7 : 73
Index 8 : 63
Index 9 : 53
Index 10 : 43
```

6.2 C Program – Quadratic Probing

```
#include <stdio.h>

#define SIZE 11

void quadraticProbing(int a[], int n)
{
    int table[SIZE];
    int i, key, index, pos, j;

    for (i = 0; i < SIZE; i++)
        table[i] = -1;

    for (i = 0; i < n; i++)
    {
        key = a[i];
        index = key % SIZE;
        j = 0;

        while (1)
        {
            pos = (index + j * j) % SIZE;

            if (table[pos] == -1)
            {
                table[pos] = key;
                break;
            }

            j++;
        }
    }

    printf("\nQuadratic Probing:\n");

    for (i = 0; i < SIZE; i++)
        printf("Index %d : %d\n", i, table[i]);
}

int main()
{
    int a[] = {23, 43, 13, 33, 53, 63, 73};
    quadraticProbing(a, 7);
    return 0;
}
```

Output

Quadratic Probing:

Index 0 : 33
Index 1 : 23
Index 2 : 13
Index 3 : -1
Index 4 : -1
Index 5 : -1
Index 6 : -1
Index 7 : 73
Index 8 : 63
Index 9 : 53
Index 10 : 43

7.2 C Program – Double Hashing

```
#include <stdio.h>
```

```
#define SIZE 11
```

```
void doubleHashing(int a[], int n)
```

```
{  
    int table[SIZE];  
    int i, key, h1, h2, pos, j;
```

```
    for (i = 0; i < SIZE; i++)  
        table[i] = -1;
```

```
    for (i = 0; i < n; i++)  
    {  
        key = a[i];
```

```
        h1 = key % SIZE;  
        h2 = 7 - (key % 7);
```

```
        j = 0;
```

```
        while (1)  
        {  
            pos = (h1 + j * h2) % SIZE;
```

```
            if (table[pos] == -1)  
            {  
                table[pos] = key;  
                break;  
            }
```

```
            j++;
```

```

    }
}

printf("\nDouble Hashing:\n");

for (i = 0; i < SIZE; i++)
    printf("Index %d : %d\n", i, table[i]);

}

int main()
{
int a[] = {23, 43, 13, 33, 53, 63, 73};

doubleHashing(a, 7);

return 0;

}

```

Output

Double Hashing:

Index 0 : 33

Index 1 : 23

Index 2 : 13

Index 3 : -1

Index 4 : -1

Index 5 : -1

Index 6 : -1

Index 7 : 73

Index 8 : 63

Index 9 : 53

Index 10 : 43